Mini Empire — Complete Documentation

All project documentation in one file (README + docs/). The individual files remain in the repository as well.

Contents

1. Overview & setup
2. Architecture
3. HTTP API
4. Economy & balancing
5. Game rules
6. Security & anti-cheat
7. Deployment
8. Roadmap

Part 1 — Overview & setup

A mobile-first medieval strategy game: build a kingdom, train an army, fight asynchronous battles, join a clan, fight for territories and compete in a 100-day season. Web first, structured for a Telegram Mini App and future Android/iOS clients.

> **Status:** playable foundation covering development phases 1–7 (see What works). Everything listed as working is implemented end-to-end and covered by automated tests or was exercised in a real browser. Known gaps are listed honestly in Limitations.

Technology choices

Layer	Choice	Why
Game rules	`shared/` — pure TypeScript, no I/O	Same rules run on the server (authoritative) and in the browser (previews, battle replays). Ready to reuse in native clients.
Server	Node 22 + Fastify 5 + `pg` + Zod	Small, fast, explicit SQL with real transactions; Zod validates every request.
Database	PostgreSQL 16, plain SQL migrations	Row locks + transactions for every economic action; CHECK

Layer	Choice	Why
		constraints as a second line of defence.
Client	React 18 + Vite 6, SVG isometric renderer	No heavy game engine: procedural SVG art scales crisply on phones and is easy to replace with illustrated assets.
Tests	Vitest (unit tests on rules, integration tests on a real PostgreSQL)	
Auth	Opaque bearer sessions (SHA-256 hashed at rest), scrypt passwords, Telegram `initData` HMAC verification	Bearer tokens work inside Telegram's web view where cookies are unreliable.

Project structure

```
mini-empire/
├── shared/                 # @mini-empire/shared — game rules & balancing (no I/O)
│   ├── src/config/         # buildings, troops, economy, missions, season, scoring, rewards, territories, PvE
│   ├── src/rules/          # kingdom (placement, upgrades, production), training, resources, periods
│   ├── src/battle/         # deterministic battle engine + seeded RNG
│   ├── src/season/         # 100-day season state machine
│   ├── src/scoring/        # leaderboards, war matchmaking, reward calculation
│   ├── scripts/balance-sim.ts  # plays a "regular player" through the real rules (days to HQ 10)
│   └── test/               # unit tests
├── server/                 # @mini-empire/server — Fastify API (authoritative)
│   ├── migrations/         # SQL migrations (schema in 001_init.sql)
│   ├── src/routes/         # auth, kingdom/army, battles, missions, clans, world, season, profile/store, admin
│   ├── src/services/       # kingdom persistence, battles, clan wars, territories, season, rewards, leaderboards
│   ├── src/auth/           # passwords, sessions, Telegram verification
│   ├── scripts/grant-role.ts
│   └── test/               # integration tests (real PostgreSQL)
├── client/                 # @mini-empire/client — React UI
│   └── src/{screens,components,render}
├── docs/                   # architecture, API, economy, game rules, security, deployment, roadmap
```

```
├── Dockerfile, docker-compose.yml,
.github/workflows/ci.yml, .env.example
```

Quick start (clean machine)

Requirements: **Node.js ≥ 20** (22 recommended) and **PostgreSQL ≥ 14**.

```
# 1. Install dependencies (npm workspaces)
npm install

# 2. Create databases (adapt user/password)
createuser -P mini            # password e.g. mini_dev_password
createdb -O mini mini_empire
createdb -O mini mini_empire_test

# 3. Configure
cp .env.example .env          # edit DATABASE_URL / TEST_DATABASE_URL
export $(grep -v '^#' .env | xargs)   # or use your preferred env
loader

# 4. Migrate (the server also migrates automatically on start)
npm run migrate

# 5. Run in development: API on :3000 + Vite dev server on :5173
(proxies /api)
npm run dev
# open http://localhost:5173 and create a kingdom
```

Production-like run (API serves the built client on one port):

```
npm run build      # builds client/dist
npm start          # http://localhost:3000
```

Or with Docker: `docker compose up --build` → http://localhost:3000.

Creating an administrator

There is intentionally **no HTTP endpoint that can create the first admin**. Register a normal account in the game, then on the server:

```
npm run admin:grant -w server -- <username> superadmin   # roles:
player | moderator | admin | superadmin
```

The admin panel is then reachable from the profile menu (tap your avatar → *Admin panel*).

Useful commands

Command	What it does
`npm test`	Unit tests (shared) + integration tests (server, needs `TEST_DATABASE_URL`; **drops & recreates that DB's schema**)

Command	What it does
`npm run typecheck`	Type-checks all three packages
`npm run balance`	Balance simulation: days for a regular player to reach each HQ level (`-- --sessions 2`, `-- --table`)
`npm run migrate`	Applies pending SQL migrations
`npm run build`	Production build of the client

Test results

Last run in the development container (Node 22.22, PostgreSQL 16.15):

```
shared  Test Files 5 passed (5)   Tests 39 passed (39)
server  Test Files 3 passed (3)   Tests 25 passed (25)
typecheck: shared, server, client — clean
client build: 273 kB JS (85 kB gzip)
```

What the tests prove (selection):

- Starting kingdom is exactly HQ 1 + one defensive tower + one wall; placement rejects overlap, out-of-bounds, water and the deploy ring.
- Upgrades need resources, HQ level, prerequisites and a free builder; costs are deducted atomically; timers complete only on server time.
- Offline production uses server timestamps, stops at the buffer cap and the 24 h window, is limited by storage, and **four concurrent collect requests credit exactly one collection**.
- Idempotency keys replay the stored response instead of executing twice; reusing a key with another body is rejected.
- Training queue: costs, per-unit completion, capacity, cancellation refunds.
- Battles are deterministic; a client sending `victory: true`, fake rewards or troops it does not own gets nothing extra; there is no endpoint to add resources or submit results; stored battles can be re-simulated and **tampering with a stored result is detected**.
- Missions: progress comes only from server events; **three concurrent claims → exactly one succeeds**; daily reset in UTC; participation capped per day.
- Clans: no double membership, role permissions (member/officer/leader), closed-clan approvals.
- Season: correct phase for every day, only forward single-step transitions, roster lock at war start, matchmaking, war attacks limited to locked rosters, scores frozen after the war phase.
- Rewards: preview → approve → distribute; re-approving/re-running distribution never pays twice; never exceeds the pool.
- Season reset: atomic, idempotent (second run is a no-op), resets kingdoms/troops/territories and **preserves** accounts, token balances, achievements, cosmetics, clan history, leaderboard snapshots and season records.

- Unauthenticated / non-admin requests are refused; public profiles leak no private fields; Telegram `initData` with a wrong signature, wrong bot or stale date is rejected.

Also verified manually in headless Chromium (390×844 phone and 1280×800 desktop): registration, onboarding, build shop, valid/invalid placement ghost, upgrades panel, campaign battle with replay and result, world map, clan creation form, season screen. The Docker image was built and run against PostgreSQL (registration and client serving worked).

What works

Phase 1 — Kingdom prototype: isometric 24×24 kingdom with pan/zoom/pinch, tap selection, building info panel, shop, placement with live validity, move, upgrade, cancel (50 % refund), level badges, construction scaffolding + timers, 10 visually distinct levels per building (wood → reinforced timber → stone → fortified masonry → royal → legendary), resource bar, season countdown, bottom navigation, onboarding.

Phase 2 — Economy & persistence: accounts (username/password, Telegram), server-side kingdom state, resource production (offline, capped), collection, storage limits, construction queue with builders, prerequisites, audit log of every balance change, idempotent mutations, row-locked transactions, data-driven balancing tables + admin tunables.

Phase 3 — Army & battles: single Barracks with unlocks, capacity, training queue, cancellation; four troop classes with target priorities and damage modifiers; deterministic tick-based battle engine with walls (path-finding cost + break-through), archer towers (nearest) and heavy towers (strongest, splash); 10 PvE stages, PvP with matchmaking, loot, shields and diminishing returns; battle reports, history and replays.

Phase 4 — Clans & world map: create/join/leave, open or approval-based joining, tag/name uniqueness, emblem, description, leader/officer/member roles with enforced permissions, kick, promote/demote, leadership transfer, polling chat, clan missions with contribution tracking, contribution stats, clan history, season results; 14-territory world map with ownership, history, capture cooldowns, attempt limits and hourly hold scoring.

Phase 5 — Seasons & rankings: 100-day server-time state machine, automatic transitions, roster selection, eligibility and lock, strength/activity matchmaking with byes, war battles and results, clan / individual / participation leaderboards with published tie-breaks, final snapshots, permanent per-player season records, atomic idempotent season reset with archives.

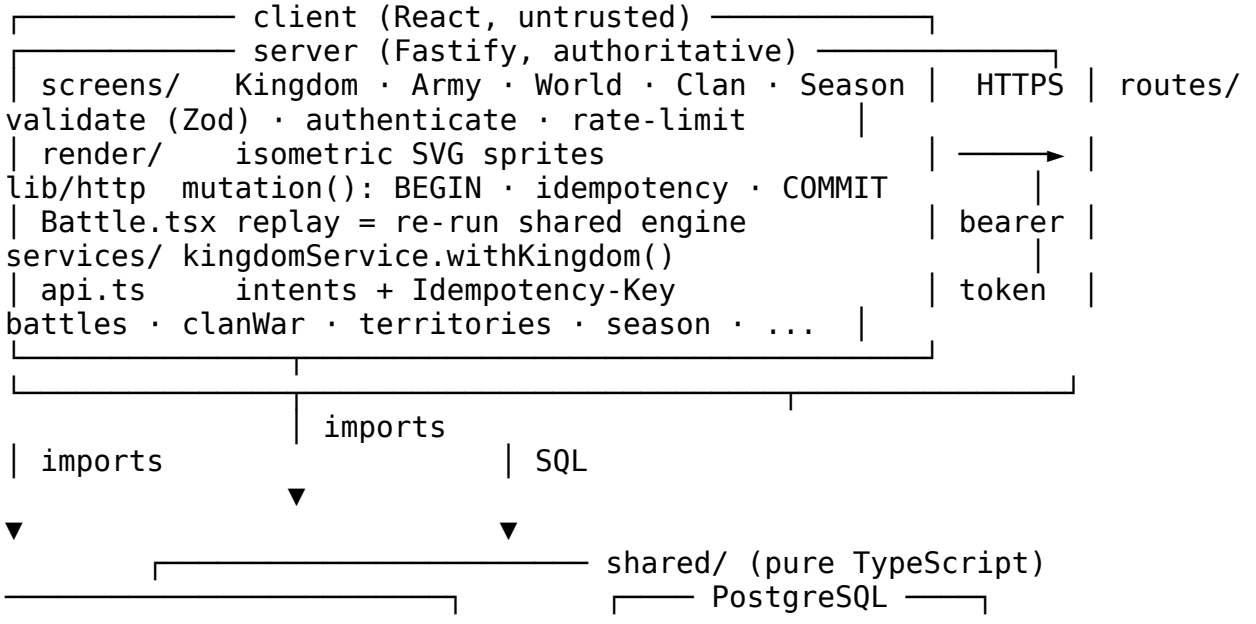
Phase 6 — Rewards & administration: simulated token ledger (pool, configurable allocation, eligibility, preview, approval, idempotent distribution, failure + retry, history), admin panel (overview/health/errors, player & clan lookup, progression inspection, balancing tunables with history, season control, war & leaderboard inspection, reward workflow, suspicious-activity review, audit log), role-based permissions.

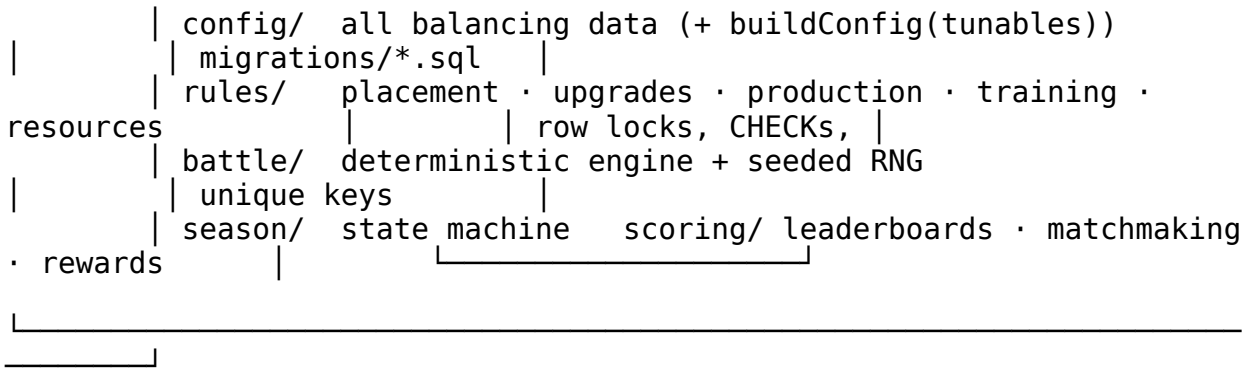
Phase 7 — Telegram & mobile readiness: verified Telegram Mini App login, safe-area aware responsive layout, Dockerfile, docker-compose, CI workflow, deployment guide, path to native clients (see docs/DEPLOYMENT.md).

Known limitations

- **Art** is procedural SVG (deliberately replaceable via `client/src/render/sprites.tsx`); no sound or music.
- **Chat** uses 4-second polling, not WebSockets.
- **Rate limiting** is in-memory (single instance). Use a shared store (e.g. Redis) before scaling horizontally.
- **Season reset** runs in one transaction over all kingdoms — fine for thousands of players; batch it for very large populations.
- **Defending troops** do not fight in battles (only towers and walls defend). Troop levels are tied to Barracks level rather than individual research.
- **Replays** re-simulate with the client's current balancing; if an operator changed balancing after a battle the replay shows a notice (the stored server result stays authoritative).
- **Store**: catalogue and inventory exist; real payments are deliberately disabled (`PAYMENTS_DISABLED`).
- **Tokens** are a simulated ledger only. There is no blockchain integration, no price and no financial promise.
- **Telegram**: server verification and client bootstrap are implemented and unit-tested with signed data, but have not been tested inside the real Telegram app (needs a bot token and public HTTPS URL).
- The balance targets HQ 10 in ~90 days for a regular player: the simulation reaches it on **day 77** with three short sessions per day and **day 86** with two (an optimal greedy bot; real players are less efficient). Tune via `docs/ECONOMY.md`.
- No e-mail accounts / password reset yet; no automated end-to-end browser test suite in CI (manual Playwright run only).

Part 2 — Architecture





Separation of concerns

Concern	Where	Notes
Game rules	`shared/src/rules`, `shared/src/battle`	Pure functions of `(state, ctx{now, cfg, newId})`. No I/O, no clock, no randomness except the seeded RNG.
Balancing data	`shared/src/config`	Tables + `buildConfig(tunables)` merges operator overrides from the `game_config` table.
Server logic	`server/src/services`	Loads state, calls rules, persists, records events.
Database access	`server/src/services/kingdomRepo.ts` and service-local SQL	Explicit SQL, parameterised.
Reward calculation	`shared/src/scoring/rewards.ts` (pure) + `server/src/services/rewards.ts` (workflow)	Integer micro-units (BigInt).
UI	`client/src`	Displays state, sends intents. Never computes authoritative outcomes.

The kingdom mutation pipeline

Every change to a player's kingdom goes through `withKingdom()` (`server/src/services/kingdomService.ts`):

1. `SELECT … FROM kingdoms WHERE user_id = $1 FOR UPDATE` — serialises all concurrent requests of a player.

2. Load buildings, construction queue, balances, troops and training queue into a `KingdomState`.
3. `settle(state, now)` — finish constructions whose timers elapsed (in time order, producers earn at the old rate until the upgrade completes) and move finished troops from the queue to the army.
4. Run the action (e.g. `startUpgrade`) — throws `GameRuleError` on any rule violation.
5. `saveKingdom()` writes only the diff; `resource_balances` has `CHECK (gold >= 0 …)`.
6. `logResourceTx()` appends the net balance change to `resource_transactions` (audit trail).
7. Rule events (`building_upgraded`, `resources_collected`, `troops_trained`) become mission progress, achievements and participation score inside the same transaction.

`mutation()` (`server/src/lib/http.ts`) wraps this in `BEGIN … COMMIT` and handles `Idempotency-Key`: the key row is inserted in the same transaction, so a retried request either waits for and returns the first response or re-executes only if the first attempt rolled back.

Time

Only the server clock is used (`ctx.now()`, injectable for tests). The client receives `serverTime` with each kingdom response and keeps an offset for countdowns. Production accrues from `last_tick_at` timestamps written by the server; the device clock is never trusted.

Battles

`simulateBattle(input, cfg)` is a 10 Hz fixed-step simulation on the 24×24 grid:

- units spawn along the chosen edges (positions from the seeded RNG), pick a target by priority (`any` → nearest building, `defense` → nearest tower), path-find with Dijkstra where walls cost 8 and other buildings 40 (so troops walk around walls when cheap, break through otherwise),
- blocked units attack the wall/building in their path; damage uses per-category modifiers (siege ×4 vs walls),
- archer towers shoot the nearest unit, heavy towers the toughest unit with splash damage,
- ends on full destruction, army wiped or 3 minutes. Stars: 50 % destruction, HQ destroyed, 100 %.

The server picks the seed, validates the deployment against owned troops, runs the simulation and stores `input` + `result` + `config_version`. The client fetches `/battles/:id` and re-runs the same engine with `record: true` to animate the replay. `/admin/battles/:id/verify` re-simulates to detect tampering.

Seasons

`seasons.state` follows `DEVELOPMENT → WAR_PREPARATION → WAR_ACTIVE → RESULT_VERIFICATION → REWARD_DISTRIBUTION → SEASON_RESET → COMPLETED`. `seasonTick()` (every `SEASON_TICK_SECONDS`, idempotent) enters the time-driven states when their day starts; entering a state runs its side effects in the same transaction (roster lock + matchmaking, war/territory finalisation + final leaderboard snapshots). Reward distribution and reset require operator actions. See GAME_RULES.md.

Database entities

The schema (`server/migrations/001_init.sql`) maps the requested entities as follows:

Requested	Table(s)
Users, PlayerProfiles	`users`, `sessions`, `player_profiles`
Kingdoms, Buildings, ResourceBalances, ResourceTransactions, ConstructionQueues	`kingdoms`, `buildings`, `resource_balances`, `resource_transactions`, `construction_queues`
Barracks, Troops, Armies	Barracks = `buildings(type='barracks')` + `training_queue`; Troops/Armies = `troops` (count per type)
Battles, BattleReports	`battles` (inputs, seed, result), `battle_reports` (per participant), `pve_progress`
Clans, ClanMemberships, ClanMessages, ClanWars	`clans`, `clan_memberships` (PK user_id ⇒ one clan), `clan_join_requests`, `clan_messages`, `clan_events`, `clan_war_rosters`, `clan_wars`
Territories, TerritoryOwnershipHistory	`territories`, `territory_ownership_history`
Missions, MissionProgress	definitions in `shared/src/config/missions.ts` (+ tunables); `mission_progress`, `clan_mission_progress`, `clan_mission_contributors`, `clan_mission_claims`
Seasons, SeasonParticipants, LeaderboardSnapshots	`seasons`, `season_transitions`, `season_participants`, `score_events`, `leaderboard_snapshots`
RewardPools, RewardAllocations,	`reward_pools`,

Requested	Table(s)
RewardTransactions	`reward_allocations`, `reward_transactions` (UNIQUE allocation_id), `token_balances`
CosmeticInventory, Achievements	`cosmetic_inventory`, `achievements`
AdministrativeAuditLogs	`admin_audit_logs`, `suspicious_events`, `error_logs`, `game_config`
(reset safety)	`season_archives`, `season_reset_log`, `idempotency_keys`

Part 3 — HTTP API

Base path: `/api`. JSON in, JSON out.

- **Auth:** `Authorization: Bearer <token>` (from register/login/telegram). Missing/invalid → `401`.
- **Idempotency:** every state-changing request may carry `Idempotency-Key: <8–80 chars [A-Za-z0-9_-]>`. A retry with the same key and body returns the stored response without executing again; the same key with a different body → `422 IDEMPOTENCY_MISMATCH`. The official client sends a fresh key on every mutation.
- **Errors:** `{ "error": { "code": "INSUFFICIENT_RESOURCES", "message": "Not enough resources (missing 200 gold)." } }`
  - `400` rule violation / invalid input (`INVALID_INPUT`, `OVERLAP`, `HQ_TOO_LOW`, `NO_BUILDER`, `NOT_ENOUGH_TROOPS` …)
  - `401` unauthenticated, `403` forbidden, `404` not found, `409` conflict (`ALREADY_CLAIMED`, `ROSTER_LOCKED`, …), `422` idempotency mismatch, `429` rate limited, `500` internal (logged to `error_logs`).
- **Rate limits** (per user, or per IP when anonymous): global 600/min/IP, kingdom actions 120/min, collect 60/min, battles 20/min, chat 10/min, register 5/h, login 10/15 min.
- Times are epoch milliseconds (server clock) unless a field is an ISO timestamp from the database.

Unknown fields in request bodies are ignored — they can never influence results.

Auth

Method & path	Body	Response
`POST /auth/register`	`{ username: 3–20 [A-Za-z0-9_], password: 8–100 }`	`{ token, user: { id, username, role } }` — creates

Method & path	Body	Response
		the account, profile and starting kingdom
`POST /auth/login`	`{ username, password }`	`{ token, user }`
`POST /auth/telegram`	`{ initData }` (raw `Telegram.WebApp.initData`)	`{ token, user }` — signature verified with `TELEGRAM_BOT_TOKEN`; 501 if not configured
`POST /auth/logout`	–	`{ ok: true }`
`GET /auth/me`	–	`{ user }`

Configuration & kingdom

Method & path	Body	Notes
`GET /config`	–	Effective balancing config (`GameConfig`): buildings (10 levels each), troops, economy, battle, missions, clan war rules.
`GET /kingdom`	–	Settles timers then returns the **KingdomView** below.
`POST /kingdom/buildings`	`{ type, x, y }`	Place a new building (charges level-1 cost, uses a builder unless instant). Response: KingdomView + `building`.
`POST /kingdom/buildings/:id/upgrade`	–	Start upgrade.
`POST /kingdom/buildings/:id/cancel`	–	Cancel construction, 50 % refund; brand-new buildings are removed. Response includes `refund`.
`POST /kingdom/buildings/:id/move`	`{ x, y }`	Free move, validated.
`POST /kingdom/collect`	`{ buildingId? }`	Collect one or all producers. Response includes `collected`.

Method & path	Body	Notes
`GET /kingdom/transactions`	–	Last 50 audit rows of the player's balance changes.
`POST /army/train`	`{ troop, count: 1–100 }`	Enqueue training (charges full cost).
`POST /army/train/:id/cancel`	–	Unstarted units 100 % refund, unit in training 50 %.

KingdomView


```
{
  "serverTime": 1791824217769, "configVersion": "default",
  "resources": { "gold": 1400, "wood": 1250, "stone": 500 },
"storageCapacity": 4000,
  "hqLevel": 1, "builders": 1,
  "buildings": [{ "id": "uuid", "type": "gold_mine", "level": 0, "x":
3, "y": 3, "stored": 0, "lastTickAt": 0,
                  "upgrade": { "toLevel": 1, "startedAt": 0, "endsAt":
0 }, "produced": 0 }],
  "troops": { "infantry": 0, "archer": 0, "cavalry": 0, "siege": 0 },
  "trainingQueue": [{ "id": "uuid", "troop": "infantry", "remaining":
5, "perUnitMs": 20000, "startsAt": 0, "unitCost":
{"gold":50,"wood":25,"stone":0} }],
  "armyCapacity": 20, "housingUsed": 5, "troopMultiplier": 1,
"shieldUntil": 0
}
```


Battles

`deployments: [{ troop: 'infantry'|'archer'|'cavalry'|'siege', count: 1–200, side: 'north'|'south'|'east'|'west' }]` (1–12 entries). Total per troop must not exceed the troops you own.

Method & path	Body	Notes
`GET /battles/pve`	–	`{ stages: [{ id, name, hqRequired, reward, firstClearBonus, bestStars, unlocked, rewardedAttemptsLeft, layout }] }`
`POST /battles/pve`	`{ stage, deployments }`	→ **BattleOutcome**
`GET /battles/opponents`	–	`{ opponents: [{ id, username,`

Method & path	Body	Notes
		`hqLevel, power, avatar, clanTag, scoreMultiplier }] }` (not shielded, not clan mates, within HQ range, not attacked 3× in 24 h)
`GET /battles/opponents/:id/layout`	–	`{ layout: [{ type, level, x, y }] }` (scouting)
`POST /battles/pvp`	`{ defenderId, deployments }`	→ BattleOutcome (loot, shield, diminishing returns)
`GET /battles`	–	`{ reports: [{ battle_id, role, summary, kind, attacker, defender, created_at }] }`
`GET /battles/:id`	–	Full record for replay (participants and staff only): `{ battle: { input, result, rewards, scorePoints, configVersion, … } }`

BattleOutcome

```
{ "battleId": "uuid", "kind": "pve", "seed": 123,
  "result": { "victory": true, "stars": 3, "destructionPercent": 100,
"hqDestroyed": true, "ticks": 382,
              "deployed": {…}, "casualties": {…}, "survivors": {…},
"buildingsDestroyed": 4, "totalBuildings": 4, "wallsDestroyed": 0 },
  "rewards": { "gold": 900, "wood": 900, "stone": 450 },
"scorePoints": 6, "notes": ["First clear bonus!"] }
```

Missions

Method & path	Notes
`GET /missions`	`{ missions: [{ id, period, title, event, target, reward, progress, claimed, resetsAt }], clanMissions: [...], serverTime }`
`POST /missions/:id/claim`	Requires progress ≥ target for the current UTC period; once only (409

Method & path	Notes
	ALREADY_CLAIMED). → { credited, resources }
POST /clan-missions/:id/claim	Clan mission completed this ISO week and you contributed; once only.

Clans

Method & path	Body	Permission
GET /clans?search=	–	any player
GET /clans/mine	–	`{ clan: ClanDetails
GET /clans/:id	–	ClanDetails: members, requests (staff only), events, seasonScore, seasonHistory, wars
POST /clans	`{ name, tag, description?, emblem: { icon, color }, joinPolicy: 'open'	'closed' }`
POST /clans/:id/join	–	not in a clan; open → joined, closed → requested
POST /clans/requests/:userId/approve / reject	–	leader/officer
POST /clans/leave	–	leader must transfer first unless last member (then the clan is disbanded)
POST /clans/members/:userId/role	`{ role: 'officer'	'member' }`
POST /clans/members/:userId/kick	–	leader (anyone), officer (members only)
POST /clans/transfer	{ userId }	leader
PUT /clans/settings	{ description?, emblem?, joinPolicy? }	leader/officer
GET /clans/chat?after=<id> / POST /clans/chat { body }		members

Method & path	Body	Permission
`GET /clans/war`	–	rules, eligibility per member, current war, enemy roster, attacks left
`PUT /clans/war/roster`	`{ userIds: uuid[] }`	leader/officer, only in DEVELOPMENT / WAR_PREPARATION
`POST /clans/war/attack`	`{ defenderId, deployments }`	locked roster member, WAR_ACTIVE, attack limit

World map

Method & path	Notes
`GET /world`	`{ season, attacksOpen, rules, myClanId, territories: [{ id, name, kind, value, mapX, mapY, adjacent, owner, status, cooldownUntil, effectiveGarrison, myAttemptsLeft, holdPoints, layout }] }`
`GET /world/territories/:id/history`	`{ ownership: [...], battles: [...] }`
`POST /world/territories/:id/attack`	`{ deployments }` → BattleOutcome; WAR_ACTIVE only, clan member, HQ ≥ 3, attempt limit, cooldown

Season & leaderboards

Method & path	Notes
`GET /season`	season (number, state, day, startsAt, endsAt, schedule), timeline, scoring explanation, my scores, simulated reward info
`GET /leaderboards/:board`	`board = clan
`GET /season/history`	last seasons with final top-10 per board

Profile & store

Method & path	Notes
`GET /profile/me`	public profile + role, onboarding flag,

Method & path	Notes
	simulated token balance
`GET /profile/:id`	public profile only (no credentials, Telegram id, IP hash, moderation flags)
`PUT /profile`	`{ avatar?, title?, equipped? }` — titles/cosmetics must be owned
`POST /profile/onboarding`	marks the tutorial as seen
`GET /store`	cosmetics catalogue + owned flags, `paymentsEnabled: false`
`POST /store/purchase`	always `501 PAYMENTS_DISABLED`

Admin (role-based; every sensitive action writes `admin_audit_logs`)

Method & path	Min role	Notes
`GET /admin/overview`	moderator	counts, season, health, recent errors
`GET /admin/players?q=·GET /admin/players/:id`	moderator	progression, balances, transactions, battles, flags, scores, same-IP accounts
`POST /admin/players/:id/review { underReview, note }`	moderator	excludes from rewards while flagged
`POST /admin/players/:id/suspend { suspended, note }`	admin	also revokes sessions
`POST /admin/players/:id/role { role, note }`	superadmin	
`GET /admin/clans?q=·GET /admin/clans/:id`	moderator	
`GET /admin/config·PUT /admin/config { tunables, note }`	moderator · admin	validated tunables, versioned
`GET /admin/season`	moderator	transitions, wars, reset log
`POST /admin/season/tran`	superadmin	forced *valid* transition

Method & path	Min role	Notes
`sition { to, reason }`		only
`POST /admin/season/schedule { startsAt, schedule, reason }`	superadmin	only before war phase
`POST /admin/season/reset { confirm: "RESET SEASON <n>" }`	superadmin	idempotent
`GET /admin/leaderboards`	moderator	live boards + snapshots
`GET /admin/rewards · POST /admin/rewards/preview { poolTokens, allocation? }`	moderator · admin	RESULT_VERIFICATION only
`POST /admin/rewards/approve`	superadmin	moves season to REWARD_DISTRIBUTION and distributes
`POST /admin/rewards/retry`	admin	re-attempts failed allocations (never double-pays)
`GET /admin/suspicious?status= · POST /admin/suspicious/:id/review { status, note }`	moderator	human decision recorded
`GET /admin/battles/:id/verify`	moderator	re-simulates stored input, `matches: true
`GET /admin/audit`	admin	last 200 audit rows

`GET /api/health` → `{ ok: true, time }` (no auth).

Part 4 — Economy & balancing

All numbers live in `shared/src/config/`. Nothing is hard-coded in the server or client.

- `buildings.ts` — 10-level table per building (cost, duration, HQ requirement, prerequisites, HP, production, buffer, storage, army capacity, tower stats, builders, unlock notes) and per-HQ building count limits.
- `troops.ts` — troop stats, costs, training time, housing, unlock level, target priority, damage modifiers.
- `economy.ts` — starting resources, offline accrual window, clan cost, cancel refunds, PvP loot, shields, repeat-attack limits, matchmaking range.
- `missions.ts`, `pve.ts`, `territories.ts`, `scoring.ts`, `rewards.ts`, `season.ts`.

Operators can adjust **tunables** at runtime from the admin panel (`PUT /api/admin/config`); they are validated, versioned in `game_config`, audited, and merged by `buildConfig()`:

```
{
  "productionMultiplier": 1.2,
  "costMultiplier": 0.9,
  "durationMultiplier": 1,
  "trainingCostMultiplier": 1,
  "trainingDurationMultiplier": 1,
  "troopHpMultiplier": { "cavalry": 0.9 },
  "troopDamageMultiplier": { "siege": 1.1 },
  "missionRewardMultiplier": 1,
  "disabledMissions": ["w_clan"],
  "battle": { "pvpLootShare": 0.1, "defenderShieldHours": 6,
"maxAttacksOnSameDefenderPer24h": 3, "matchmakingHqRange": 2,
"minSecondsBetweenBattles": 5 },
  "clanWar": { "minHqLevel": 5, "minMembershipHours": 48,
"minRosterSize": 3, "maxRosterSize": 20, "attacksPerMember": 3 }
}
```

Structural changes (new levels, prerequisites, unlocks) are edited in code and shipped with a release.

Resource rules (enforced by code and database)

- Three resources: gold, wood, stone. Starting balance 1 500 / 1 500 / 500.
- Balances can never be negative: `spend()` checks before deducting; `resource_balances` has `CHECK (x >= 0)`.
- Costs are deducted inside the same locked transaction that changes the building — all-or-nothing.
- Producers fill an internal buffer (`bufferCap` = 10 h of production). Production stops when the buffer is full; offline time is never computed beyond `maxOfflineAccrualHours` (24 h) since the last collection.
- Collection is limited by storage capacity (`HQ storageCap + Σ Storage buildings`); what does not fit stays in the producer. Rewards (missions, battles) are also capped by storage — the UI warns when storage is full.
- Every balance change writes a `resource_transactions` row with delta, balance after, reason and reference.

Headquarters (progression spine)

HQ level	Gold	Wood	Stone	Duration	Storage	Builders	Requires
1	0	0	0	–	4000	1	-
2	800	800	0	5 min	8000	1	-
3	3000	3000	1500	1 h	15000	2	barracks 1
4	9000	9000	6000	4 h	25000	2	storage 2
5	20000	20000	15000	10 h	40000	2	archer_tower 3
6	42000	42000	34000	18 h	60000	3	barracks 4
7	80000	80000	66000	1 d	90000	3	storage 5
8	140000	140000	120000	2 d	130000	3	heavy_tower 5
9	250000	250000	220000	3 d	180000	3	barracks 7
10	380000	380000	330000	4.5 d	240000	3	storage 8

Other buildings: level *n* requires HQ ≥ *n* − *1* (Heavy Tower: HQ ≥ 3). Costs grow ×1.95 per level (×1.8 walls), durations follow `10 s, 2 m, 15 m, 1 h, 3 h, 6 h, 10 h, 16 h, 1 d, 1.5 d` (walls: instant level 1, ≤ 1 h).

Producers (Gold Mine shown; Lumber Camp / Stone Quarry identical output, different cost mix)

Lvl	Cost (g/w/s)	Time	HQ	HP	Output
1	100/250/0	10s	1	400	150/h, buffer 1500
2	200/490/0	2m	1	472	191/h, buffer 1910
3	380/950/0	15m	2	557	242/h, buffer 2420
4	740/1900/0	1h	3	657	307/h, buffer 3070
5	1400/3600/0	3h	4	776	390/h, buffer

Lvl	Cost (g/w/s)	Time	HQ	HP	Output
					3900
6	2800/7000/0	6h	5	915	496/h, buffer 4960
7	5500/14000/0	10h	6	1080	629/h, buffer 6290
8	11000/27000/0	16h	7	1274	799/h, buffer 7990
9	21000/52000/0	1d	8	1504	1015/h, buffer 10150
10	41000/100000/0	1.5d	9	1774	1289/h, buffer 12890

Max producers of each type by HQ level: 1,1,2,2,2,3,3,3,4,4. Storage buildings: 1,1,1,1,2,2,2,3,3,3 (+3 000 → +210 000 capacity per building).

Troops

Troop	HP	Dmg	Interval	Speed	Range	Cost (g/w/s)	Train	Housing	Barracks
Infantry	130	14	1s	1	0.6	50/25/0	20s	1	1
Archers	60	11	1s	1.1	3.5	70/40/0	25s	1	2
Cavalry	300	24	1.2s	2	0.6	160/0/60	60s	3	4
Siege Ram	450	60	3s	0.6	0.6	300/300/120	180s	5	6

Barracks army capacity: 20, 30, 40, 55, 70, 85, 100, 120, 140, 160. Troop HP/damage +6 % per Barracks level above 1. Damage modifiers: archers ×0.5 vs walls; cavalry ×1.2 vs towers; siege ×4 vs walls, ×2 vs towers, ×0.5 others.

Defences

- Archer Tower: 24 dmg × 1.22^(L–1) every 0.9 s, range 6, targets nearest. Max 1–5 by HQ.
- Heavy Tower: 70 dmg × 1.22^(L–1) every 2.5 s, range 5, splash 1.2, targets toughest. From HQ 3, max 1–3.

- Walls: 300 HP × 1.38^(L−1); 8 → 90 segments by HQ level. Path-finding cost 8 tiles, so armies break walls only when walking around is longer.

Battle & mission income

- Campaign: 10 stages; reward = stage reward × (0.4 / 0.7 / 1.0 for 1/2/3 stars), first-clear bonus ×2, max 3 paid attempts per stage per UTC day.
- PvP loot: 12 % × destruction % of the defender's stored balance, capped at 6 000 × attacker HQ per resource; the defender loses exactly what the attacker receives. Defender shield 4 h.
- Missions: 4 daily (≈ 1 050 gold, 850 wood, 350 stone total) and 4 weekly missions.

The 90-day target — balance simulation

`npm run balance` plays a "regular player" through the real rules: a few short sessions per day, collecting, building new producers when allowed, upgrading the HQ as soon as possible, otherwise its prerequisites, storage, then producers; it also claims the main daily/weekly missions. Current results:

Sessions per day	HQ 5	HQ 8	HQ 10
3 (8:00, 13:00, 20:00)	day 9	day 46	**day 77**
2 (8:00, 20:00)	day 9	day 54	**day 86**
1 (19:00)	day 16	day 95	not reached by day 100

The bot plays optimally (greedy, never idle when it can afford something), so real "regular" players land around the 90-day target; once-a-day players finish the season around HQ 8–9, which keeps the season competitive without demanding hours of play per day.

How to tune

1. Change a seed in `buildings.ts` (e.g. production growth `1.27`, HQ costs) or use tunables for quick global changes (`productionMultiplier`, `costMultiplier`, `durationMultiplier`).
2. Run `npm run balance` (and `npm run balance -- --sessions 2`) until HQ 10 lands near day 85–90.
3. Run `npm test` — content tests assert monotonic tables and valid PvE/territory layouts.
4. For live servers, prefer tunables (audited, reversible). A tunables change takes effect immediately for new actions; running timers keep the duration they were started with.

Part 5 — Game rules

Kingdom

- 24×24 isometric grid. The outer ring is the attackers' deploy zone; a pond and a rock outcrop are unbuildable.

- New players start with exactly: **Headquarters 1**, **Archer Tower 1**, **one Wall 1** and 1 500 gold, 1 500 wood, 500 stone.
- Buildings: Headquarters, Gold Mine, Lumber Camp, Stone Quarry, Storage, Barracks, Archer Tower, Heavy Tower, Wall. Each has 10 levels with distinct visuals (wood → reinforced timber → stone → fortified masonry → royal → legendary).
- Construction uses a builder (1 → 3 builders with HQ level); walls are built instantly at level 1.
- Buildings can be moved for free at any time; constructions can be cancelled for a 50 % refund.

Battles

- Asynchronous: the defender never needs to be online. Only towers and walls defend.
- Choose how many of each troop to send and from which side. The server picks the random seed and resolves the battle with the deterministic engine; the client then replays it.
- Stars: ★ 50 % destruction (walls excluded), ★ Headquarters destroyed, ★ 100 % destruction. Victory = ≥ 1 star.
- Fallen troops are lost; survivors return. Battles last at most 3 minutes.
- **Campaign (PvE):** 10 stages, each unlocked by the previous stage and an HQ level. Rewards for the first three attempts per stage per UTC day.
- **Player battles (PvP):** opponents within ±2 HQ levels, not in your clan, not shielded. Loot is taken from the defender's stored resources. The defender gets a 4 h shield. Repeat attacks on the same player within 24 h give 100 % / 50 % / 25 % of the points and are refused after the third.

Clans

- One clan per player (enforced by the database). Name and tag are unique (case-insensitive).
- Roles: **leader** (everything), **officer** (approve requests, kick members, edit settings and the war roster), **member**. Only the leader promotes/demotes or transfers leadership; the leader cannot leave without transferring (unless they are the last member, which disbands the clan). Max 30 members.
- Clan missions: weekly shared goals fed by members' validated actions; every member who contributed can claim.
- Clan history (joins, promotions, wars, captures, season results) is permanent.

Clan wars

Before the war (days 1–93): leaders/officers choose a roster of eligible members.

- Eligible: HQ ≥ 5, member of the clan for ≥ 48 h at roster lock, not suspended. Roster size 3–20.

Roster lock (start of day 94, automatic):

1. Unlocked roster entries are re-validated; ineligible entries are removed and logged in the clan history.

2. Clans with fewer than 3 valid players are not entered (logged).
3. Remaining entries are locked with a power snapshot. No roster changes are possible afterwards — rosters are the official record.
4. Matchmaking: each clan's rating = average member power × (0.5 + 0.5 × share of members active in the last 7 days) × √roster size. Clans are sorted by rating and paired with their neighbour, so strength and activity matter, not just size. With an odd number of clans the lowest-rated gets a **bye** (no points, no loss).

War (days 94–98): each roster member has 3 attacks against the enemy roster. Only the **best** result against each enemy player counts (10 points per star).

Result (start of day 99): more points wins; tie → higher total destruction; still tied → draw. Win = +150 clan score, draw = +50 (a war where neither side scored is a "no contest": no bonus). Inactive roster players simply contribute nothing (a fully inactive clan effectively forfeits). Disputes: every war battle stores its seed and inputs and can be re-simulated by staff (`/admin/battles/:id/verify`); decisions are audited.

Territories

- 14 territories (neutral, resource, strategic, event) with value 1–4 and an NPC garrison.
- Attackable only during the war phase, by clan members with HQ ≥ 3; 2 attempts per player per territory per day.
- ≥ 2 stars captures the territory for your clan. A captured territory is protected for 6 h and its garrison gets +1 level while owned.
- Scoring: capture = 40 individual points + 60 × value clan points; holding = 2 × value clan points per hour during the war phase (credited once, at the end of the war phase).

The 100-day season

Days	State	What happens
1–90	`DEVELOPMENT`	Build, battle, missions; rosters can be prepared
91–93	`WAR_PREPARATION`	Final roster selection
94–98	`WAR_ACTIVE`	Rosters locked + matchmaking at the start; clan wars and territory battles
99–100	`RESULT_VERIFICATION`	Wars/territories finalised, final leaderboard snapshots, scores frozen, anti-cheat review, reward preview
–	`REWARD_DISTRIBUTION`	Entered only when a superadmin approves the reward preview
–	`SEASON_RESET` → `COMPLETED`	Operator-triggered reset; the next season

Days	State	What happens
		starts in `DEVELOPMENT`

- Times are server UTC. Transitions are only ever single forward steps; invalid transitions are rejected.
- The schedule is configurable per season, but a published season keeps its schedule unless a superadmin changes it with a reason (audited) — and only before the war phase.
- If verification takes longer than day 100 the season simply waits in `RESULT_VERIFICATION`; nothing is distributed or reset automatically.

Season reset

One database transaction, idempotent (`season_reset_log` has one row per season):

- **Backed up** first: every kingdom (balances, buildings, constructions, troops, training) into `season_archives`.
- **Reset:** buildings, resources, troops, training/constructions, territory ownership, clan contribution counters. Every kingdom returns to the starting layout.
- **Preserved:** accounts, profiles and battle record, verified (simulated) token balances, cosmetics, achievements and titles, clan membership and history, season participant records, leaderboard snapshots, battle history, audit logs.
- Season rewards granted in the same transaction: "Seasoned" achievement (HQ ≥ 3) and the Champion Crown frame for members of the top clan.

Scoring & leaderboards

Points exist only as `score_events` rows created by the server, each with a unique source reference — the same battle, mission or capture can never be counted twice. Scores stop counting when the season leaves `WAR_ACTIVE`.

Board	Sources
Individual (battle)	10 pts/star PvP (with repeat-attack multiplier), 15 pts/star war, 2 pts/star campaign (rewarded attempts only), 10 pts/star territory battles, +40 per capture
Clan	10 per member war star (best per defender) + win/draw bonus, 60 × value per capture, hold points, 50 per clan mission, 25 % of members' battle points
Participation	5 per active day, mission claims (5 daily / 20 weekly), **capped at 40 per UTC day**

Tie-breaks (published in-game): higher score → earlier time of last scoring event → older account/clan.

Rewards (simulated)

The game never promises token prices, profits or returns. The reward ledger is a **simulation**:

- Pool and allocation are configurable per season (default illustrative split: 40 % clan, 30 % individual, 20 % participation, 10 % special achievements & fair play).
- Eligibility: HQ ≥ 3 at verification, not suspended, not under review. Clan pool: top 20 clans by score, split by score, then inside each clan by member contribution. Individual pool: top 100 by battle score. Participation: participation score ≥ 20. Special: special achievements earned this season with no actioned anti-cheat case.
- Workflow: preview (admin) → approve (superadmin) → distribute. Each allocation has at most one ledger transaction (unique key), so distribution can be re-run safely; failures (e.g. account suspended after approval) are recorded and can be retried. Rounding always goes down; the remainder stays in the pool.

Part 6 — Security & anti-cheat

The client is treated as untrusted. It sends **intents** ("upgrade building X", "attack stage 3 with these troops"); the server derives every outcome.

Threat	Mitigation	Where
Creating resources	No endpoint accepts amounts. Production is computed from server timestamps; rewards are computed server-side and capped by storage.	`shared/src/rules/kingdom.ts`, `services/battles.ts`
Negative balances / overspending	`spend()` checks first; `CHECK (gold >= 0 …)` in the DB; all-or-nothing transactions.	`rules/resources.ts`, `001_init.sql`
Double requests / races	Per-kingdom row lock (`FOR UPDATE`); `Idempotency-Key` stored in the same transaction; unique constraints for claims, score events, ledger rows. Tested with concurrent requests.	`services/kingdomService.ts`, `lib/http.ts`
Faking a victory	Server chooses the seed, validates troops, runs	`services/battles.ts`,

Threat	Mitigation	Where
	the simulation, stores input + result; body fields like `victory`/`rewards` are ignored. Admins can re-simulate to detect tampering.	`/admin/battles/:id/verify`
Finishing timers early	Completion only when server time ≥ `ends_at`; device clock never used.	`settle()`
Claiming twice	Row lock + `claimed_at` check (missions); `INSERT … ON CONFLICT DO NOTHING` (clan missions); `UNIQUE (allocation_id)` (rewards).	`routes/missions.ts`, `services/rewards.ts`
Point farming	Unique `source_ref` per score event; repeat-attack diminishing returns and limits; shields; PvE paid attempts/day; territory attempts/day + cooldown; participation daily cap; min seconds between battles.	`services/scores.ts`, config
Multi-accounts	Salted IP hash on registration → `many_accounts_same_ip` signal; admin view of same-IP accounts. Clan mates can't attack each other.	`routes/auth.ts`, admin
Unauthorized clan actions	Role checked after locking the clan row; membership PK prevents duplicates; leader uniqueness index.	`routes/clans.ts`
Roster manipulation	Edits only before the war; lock re-validates eligibility and is final;	`services/clanWar.ts`

Threat	Mitigation	Where
	war attacks require locked roster membership on both sides.	
Privilege escalation	Role hierarchy enforced server-side per route; first admin only via CLI; role changes audited.	`lib/http.ts`, `scripts/grant-role.ts`
Account takeover	scrypt (N=16384) password hashes, timing-safe compare; opaque random 256-bit tokens stored as SHA-256; sessions expire; suspension revokes sessions; login/register rate limits.	`auth/`
Spoofed Telegram identity	HMAC-SHA256 verification of `initData` with the bot token, freshness check (24 h). Client-supplied ids are never trusted.	`auth/telegram.ts`
Data leakage	Public profile endpoint selects an explicit safe field list (tested). No emails stored. Raw IPs never stored.	`routes/profile.ts`
Injection	Parameterised SQL everywhere; Zod validation of every body/param; 64 KB body limit.	routes
Abuse / DoS	Per-IP global limit, per-user bucket limits on expensive routes.	`lib/rateLimit.ts`

Risk-based review (no automatic bans)

Signals are written to `suspicious_events` (idempotency key reuse with a different body, many accounts per IP, …). Moderators review them in the admin panel and record a decision (`dismissed` / `actioned`) with a note; flagging a player "under review" excludes them from reward eligibility until cleared; suspension requires an admin and a note. Every decision is in `admin_audit_logs`.

Audit trails

- `resource_transactions` — every balance change (delta, balance after, reason, reference).
- `battles` — seed, inputs, config version, result, rewards, score.
- `score_events` — every point with its source.
- `season_transitions`, `season_reset_log`, `season_archives`.
- `reward_allocations`, `reward_transactions` — full payout history including failures.
- `admin_audit_logs` — config changes, forced transitions, reschedules, reward approvals, resets, moderation.

Backups & recovery

- Season reset is a single transaction (all-or-nothing) and archives every kingdom first.
- Run `pg_dump` before every season reset and on a schedule (see DEPLOYMENT.md); test restores regularly.
- A reset can be re-run safely; it never double-applies.

Token rewards — future integration

The ledger is simulated. A real token integration must be a separate, reviewed service: keys in an HSM/KMS (never in the frontend or this database), signed payout requests referencing `reward_transactions.id` as the idempotency key, transaction monitoring and reconciliation, and legal/compliance review (KYC, jurisdictions, tax). No price, profit or return is promised anywhere in the game.

Part 7 — Deployment

Single server (simplest)

1. PostgreSQL 14+ with a dedicated user and database.
2. Environment (see `.env.example`): `DATABASE_URL`, `IP_HASH_SALT` (required in production), optional `TELEGRAM_BOT_TOKEN`, `PORT`, `FIRST_SEASON_START`.
3. Build and start:

   ```
   npm ci
   npm run build          # client/dist
   NODE_ENV=production npm start
   ```

   The server applies migrations on start, creates season 1 if none exists, serves the client and the API on one port, and advances the season clock every `SEASON_TICK_SECONDS`.
4. Put it behind HTTPS (nginx, Caddy, a cloud load balancer) and set `TRUST_PROXY=true` so client IPs come from `X-Forwarded-For` (leave it `false` when the server is exposed directly, otherwise IPs could be spoofed).
5. Create the first admin: `npm run admin:grant -w server -- <username> superadmin`.

Docker

```
POSTGRES_PASSWORD=... IP_HASH_SALT=... docker compose up --build -d
```

The image builds the client and runs the TypeScript server with `tsx`. (Verified: built and run against PostgreSQL 16 during development.)

Scaling notes

- The API is stateless except the in-memory rate limiter and the season tick. For several instances: move rate limiting to Redis, and run the season tick on all instances (it is idempotent and uses row locks) or on one.
- All economic writes use row locks, so horizontal scaling does not weaken consistency.
- Season reset is one transaction over all kingdoms; batch it (still idempotent per kingdom) for very large player counts.

Backups

- Nightly `pg_dump -Fc` with retention (e.g. 14 daily, 8 weekly), stored off-host and encrypted.
- **Always** take a dump right before approving rewards and before the season reset.
- Restore drill: `pg_restore -d mini_empire_restore dump.file`, start a server against it, check `/api/health`.

Telegram Mini App

1. Create a bot with @BotFather, set `TELEGRAM_BOT_TOKEN` on the server.
2. Deploy the game on a public HTTPS URL and configure it as the bot's Mini App / menu button URL.
3. When opened inside Telegram, the client loads `telegram-web-app.js`, sends the raw signed `initData` to `POST /api/auth/telegram`, and the server verifies the HMAC signature and freshness before creating or finding the account (linked by Telegram user id, never by the client-reported name).
4. Layout already respects safe areas and works at 360 px width; no cookies are needed (bearer tokens).

Not yet verified inside the real Telegram client (requires a bot and public URL).

Path to Android and iOS

- **Fastest:** wrap the web build with Capacitor (same React UI, add store-compliant login: Sign in with Apple / Google mapped to new `/auth/*` providers using the same session mechanism).
- **Native:** reuse `@mini-empire/shared` (pure TypeScript rules, battle engine for replays) in React Native, or port the deterministic engine; the server API is client-agnostic.
- Before selling anything in-app: implement store receipt verification on the server, refunds/chargebacks handling and the platform rules — the store module is deliberately payment-free today.

Part 8 — Roadmap

Ordered by player impact vs. risk.

1. **Telegram field test** — run the Mini App inside Telegram with a test bot; polish touch gestures and haptics; add deep links to clan invites.
2. **Art & audio pass** — replace procedural sprites (`client/src/render/sprites.tsx`) with illustrated tiles keeping the same API; battle effects and sound.
3. **Defending troops & troop research** — a garrison from the Barracks that defends, and per-troop upgrades (engine already supports stat multipliers).
4. **Live updates** — WebSockets/SSE for clan chat, war score and incoming-attack notifications (currently polling).
5. **Speed-ups and builder economy** — optional (non-purchasable at first) speed-ups from missions to reduce the hard dependence on being online at timer end.
6. **Automated E2E tests** — Playwright suite in CI for registration, building, battle replay and clan flows.
7. **Operational hardening** — Redis rate limiting, structured logs/metrics, alerting on `error_logs`, batched season reset, archive restore tool.
8. **Account recovery** — optional e-mail or platform login linking, password reset.
9. **Store & season pass** — only after platform payment requirements, receipt verification, refunds and legal review; cosmetics only, no pay-to-win.
10. **Token integration** — separate, audited payout service (see SECURITY.md); keep the simulated ledger as the source of truth for allocations.

Balancing backlog

- Watch real telemetry for days-to-HQ-10 (target ~90) and adjust `productionMultiplier` / HQ costs.
- Late campaign stages (8–10) are tuned for HQ 7+ armies; check clear rates.
- Territory garrisons 1–3 are easy by design (neutral land); strategic territories may need +1 garrison level.